The terminal, and how to read what Claude does

Florian Gärtner

Where we are today

	What this course is for
	Getting it running
	Try it out
→	**The terminal, and how to read what Claude does** ← you are here
	Lunch
	A working day with Claude Code
	What the model reads, predicts and weighs
	A model, a menu, and a loop
	It doesn’t remember. It’s handed everything again.
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Tell apart** the terminal, the shell and the programs that run in it.
2. **Explain** why Claude runs commands rather than clicking.
3. **Read** a command Claude ran and say what it did.

You have all seen this

```
● Bash(git init)
  ⎿  Initialized empty Git repository in …/my_project/.git/

● Bash(git add .gitignore CLAUDE.md clean.R … && git commit -m "…")
  ⎿  Error: Exit code 128
     error: unable to write file .git/objects/e4/08b2…: Filename too long

● Bash(git config --global core.longpaths true)
  ⎿  Done

● Bash(git add .gitignore CLAUDE.md clean.R … && git commit -m "…")
  ⎿  [main (root-commit) 711624c] Initialize project repository with source files
      7 files changed, 210 insertions(+)
```

Three questions:

- What is that window?
- Why does Claude run commands at all?
- Should I be worried about what Claude types?

1. What is that window?

Not one thing. Three.

Terminal — the window. Draws text, takes keystrokes. Understands nothing.

Shell — the program inside it. Reads what you type and starts things.

Programs — Python, R, Git, Claude Code. They run in the window too.

The window is older than the computer on your desk



A DEC VT100. Photo: Jason Scott, CC BY 2.0, Wikimedia Commons.

A screen and a keyboard. No computer inside.

Wired to a machine in another room.

What you open today is an emulator of that.

There is more than one shell

cmd — the old Windows one. Still runs `.bat` files.

PowerShell — the modern Windows one.

bash — the Unix one. What Linux and most servers run.

Three shells. Not three terminals. All three can run in the same window.

Everyday commands in bash and PowerShell

	bash	PowerShell
where am I	`pwd`	`pwd`
list files	`ls`	`ls`
change folder	`cd`	`cd`
show a file	`cat`	`cat`
find text in files	`grep`	`Select-String`
compare two files	`diff`	`Compare-Object`
copy	`cp`	`cp`
move or rename	`mv`	`mv`
delete	`rm`	`rm`

There is also cmd, the old Windows shell, with names of its own.

How to read these slides: a dark box is a command that runs in a terminal, like `claude`. A violet one is typed inside Claude Code, like `/cd`.

Which shell Claude uses

Your computer	Claude's commands run in
Mac	your usual shell (zsh or bash)
Windows with Git	bash, which Git brings along, and PowerShell
Windows without Git	PowerShell

The screen tells you which: `Bash(…)` or `PowerShell(…)`.

Why bash comes first

Most command examples online are written for bash: tutorials, READMEs, answers on forums.

Claude learned from that text, so bash is most likely the shell Claude knows best.

Your desktop is a shell too

The desktop, the taskbar, the Start menu: that is the **GUI**, the graphical user interface. One program draws all of it, **Explorer**.

To start a program	you	and this program starts it
on the desktop	double-click its icon	Explorer
in the terminal	type its name	the shell

Windows says so itself. In **PowerShell** (not bash):

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v Shell

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
    Shell    REG_SZ    explorer.exe
```

TERMINAL

2. Why does Claude run commands at all?

Because a command says what it does

A click is a spot on the screen. “Click at 412, 230” means nothing once the window has moved.

A command is a sentence that names the action. `git init` does the same on any machine, on any day.

What a sentence buys you

- save it
- run it again
- put it in a script
- write it into a log
- compare two of them
- send it to someone else

Not everything is a command

For files, Claude Code has tools built in: to read a file, write one or edit one. Claude asks, and Claude Code does it, with no shell involved.

Commands are for running things, such as your R script, Python, git or installing a package.

The screen names the tool on every line: `Read(…)`, `Write(…)` or `Update(…)` for a file, `Bash(…)` or `PowerShell(…)` for a command.

Every line is Claude typing

```
● Bash(git config --global core.longpaths true)
  └  Done
```

Part	What it is
`Bash`	Claude Code’s tool that passes a command to a shell
`git config --global core.longpaths true`	the command, exactly as you would type it
`└`	what came back

Not your window: Claude Code starts a shell of its own, out of sight.

Same commands you could type.

Your R code runs as a script

Claude does not use the R console but runs your code in one go, `Rscript analysis.R`, and reads what the script prints.

- Every command starts fresh: R does not remember the one before
- Plots go to files, never to a window, so Claude saves them and opens the file to look
- Python works the same way. With the kit, Claude runs it through uv: `uv run analysis.py`

3. Should I be worried about what Claude types?

Back to the screen we started with

Claude was turning a project folder into a git repository. On the way, Claude ran:

```
● Bash(git config --global core.longpaths true)
  └  Done
```

`--global`: the setting now applies to every git repository you use on this computer.

Claude's report at the end:

> Your git repository is set up. Here's what I did:
> ✓ Copied all source files to the local folder …
> ✓ Created a .gitignore that tracks only your source files …
> ✓ Initialized git and committed all source files
>
> CLAUDE

Nothing about the setting. Only the screen shows it.

You can read what Claude types

```
● Update(clean.R)
  └  Added 1 line, removed 1 line
       misallocation <- 0.25
       misallocation <- 0.30

● Read(estimate.R)
  └  Read 40 lines
```

- An edit shows the lines it changed. A command shows what it printed.
- A read shows only which file Claude read, not what was in it.
- `Ctrl+O` shows each edit and each output in full. A read stays one line even then.

Reading it is the job now.

Two questions that turn trust into checking

Claude says the variable is defined in the setup script. Ask:

> Search for where `misallocation` is defined, and show me the line.

Claude changed four files. Ask:

> Show me exactly what you changed, line by line.

Claude runs the search and the comparison for you. Read what they printed (`Ctrl+O` shows all of it), not Claude's sentence about it.

When Claude Code asks

```
 Bash command
 Run a one-line Node.js calculation
----------------------------------------
 node -e "console.log(1 + 1)"
----------------------------------------
 Do you want to proceed?
 ❯ 1. Yes
   2. Yes, and don’t ask again for: node *
   3. Yes, and switch to auto mode · …
   4. No
```

- The line above the box is Claude’s description. The box is what runs, so read the box.
- Option 2 lets every later `node` command in this project run without asking, in later sessions too. One such rule for `git` let the `--global` change through.
- Not sure what a command changes? Choose No and ask Claude.
- **Your kit install is asking you to approve its changes right now.** Read each one, and stay in Manual until the install is done.

And every line can be stopped before it runs

Claude acts only through lines like these: `Bash(…)`, `Update(…)`, `Read(…)`.

Claude Code checks each one before it runs. Your settings and the permission mode decide which ones run at once, which ask you first, and which never run.

That is where permissions and hooks sit. The settings your Claude is merging right now hold all three: what runs at once, what asks you first (a delete command), what never runs.

4. When you start Claude Code

When the shell cannot find Claude Code

Same problem, three wordings:

```
bash: claude: command not found
```

```
The term 'claude' is not recognized as the name of a cmdlet, function,
script file, or operable program.
```

```
'claude' is not recognized as an internal or external command
```

The file exists. The shell still cannot find it.

PATH is the list of folders the shell searches. If the program is not in one of them, it does not exist as far as the shell is concerned.

PATH is an environment variable

PATH is one of the named values your system hands to every program it starts. Your home folder is another.

```
echo $PATH
```
TERMINAL

```
$env:Path
```
TERMINAL

Each program gets its own copy when it starts. A change reaches only programs started after it, which is why you open a new terminal.

Putting Claude Code’s folder on PATH

The installer puts `claude` in a folder of its own. If the shell cannot find `claude`, add that folder to PATH once.

Windows | macOS

1. Start menu: search **environment** (German: **Umgebungsvariablen**), open “Edit the system environment variables” (“Systemumgebungsvariablen bearbeiten”), then click **Environment Variables…**.
2. In the upper list, your own, select **Path**, then **Edit**.
3. **New**, paste `%USERPROFILE%\.local\bin`, then **OK** in each window.
4. Close every terminal window, open a new one and type `claude --version`.

A version number means the shell finds Claude Code now.

The same can happen to R and Python

On Windows, R's installer does not put R on PATH. The shell then cannot find `Rscript`, and Claude has to look for R itself.

By then Claude Code is running, so ask Claude:

> Find where R is installed and add its `bin` folder to my PATH.

Then restart the program the terminal runs in (Positron, VS Code or PowerShell) and Claude Code.

PATH can also name an old R. Installing a newer one leaves PATH as it was, so Claude keeps running the old version. Ask Claude:

> Which R version do you run, and is a newer one installed?

Python installed with uv is on PATH only under a versioned name, such as `python3.13`. Your kit has Claude run Python through `uv run`, which avoids this.

Your launcher is environment variables

The file you start Claude Code with sets three of them, then starts it:

Line	What it does
`ANTHROPIC_BASE_URL`	the address: the university's server instead of Anthropic's
`ANTHROPIC_AUTH_TOKEN`	the key: your budget at the HRZ
`ANTHROPIC_API_KEY`, emptied	removes any other key your computer may have set, so only yours is in play
`claude %*` (Mac: `exec claude "$@"`)	starts Claude Code and passes on whatever you type after the file's name

The last line is why this works, the next time you start Claude Code from a terminal in your work folder:

```
1  claude-jlu --resume
```
TERMINAL

```
1  claude-jlu.command --resume
```
TERMINAL

The address says where, the key says who pays

The idea to remember

The address says where the request goes. The key says who pays.

- Set only the address, and your subscription still pays.
- Set a key, and it outranks your subscription, even while you are logged in.
- Claude Code reads both when it starts. That is why, at the five-hour wall, you leave Claude Code and start the launcher again.

Another address, another model

Demo

My second launcher points the same lines at OpenRouter, a service that forwards to many model providers:

```
set ANTHROPIC_BASE_URL=https://openrouter.ai/api
set ANTHROPIC_AUTH_TOKEN=sk-or-...
claude --model moonshotai/kimi-k3 %*
```

Claude Code then runs on Kimi K3, an open-weights model from Moonshot AI, with the same instructions, files and tools.

Swapping the model is a change of configuration. Where your data goes changes with it: here to OpenRouter, and on to whichever provider serves Kimi K3.

Set up your kit

The next ten minutes are for your kit. Claude's answer is in the session you started for it.

- Read what Claude suggests adopting, and decide what you want.
- Tell Claude what to take. Claude asks before each change: read it, then approve it.

Not done by lunch? Leave the session open and finish after lunch. The kit takes effect the next time you start Claude Code.

Done early? Ask Claude for something in your project that needs a command, and read the command before you approve it.